

Project Report

Author

Name: Rewa Kiran Shete

Roll No: 23F3001275

Email: 23f3001275@ds.study.iitm.ac.in

About Me: I am a third-year engineering student. I love learning new things. Though not greatly experienced, I like creating innovative and useful websites as well as experimenting with AI.

Description

The problem statement requires us to create a multi user CRUD application that provides a one-stop interface to manage various parking lots, spots and reservations.

Technologies Used

The following technologies and libraries were used to develop the application. Each was chosen for a specific purpose to streamline development, ensure maintainability, and enhance the user experience.

Backend

> **Flask**

A lightweight Python web framework used to handle HTTP requests, manage routes, and serve as the foundation for the backend logic.

Purpose: Enables rapid development of web applications with minimal boilerplate while providing flexibility for custom business logic.

> **Flask Extensions**

Flask-SQLAlchemy : Integrates SQLAlchemy ORM with Flask for seamless database operations.

Purpose: Simplifies handling of relational databases through Pythonic ORM patterns.

Flask-Migrate & Alembic: Provide database migration support.

Purpose: Allow for version control and incremental changes to the database schema.

> **SQLAlchemy**

A powerful database toolkit and ORM for Python.

Purpose: Facilitates interaction with the database using object-oriented methods instead of raw SQL.

> **SQLite**

A lightweight embedded database.

Purpose: Stores application data without requiring setup of a separate database server, making development and testing easier.

Frontend

> HTML & CSS

Standard markup and style sheet languages for structuring and styling web pages.

Purpose: Define the content and appearance of each page for the user interface.

> Bootstrap

A widely used CSS framework.

Purpose: Provides a responsive grid system and pre-designed UI components, ensuring consistent and mobile-friendly layouts.

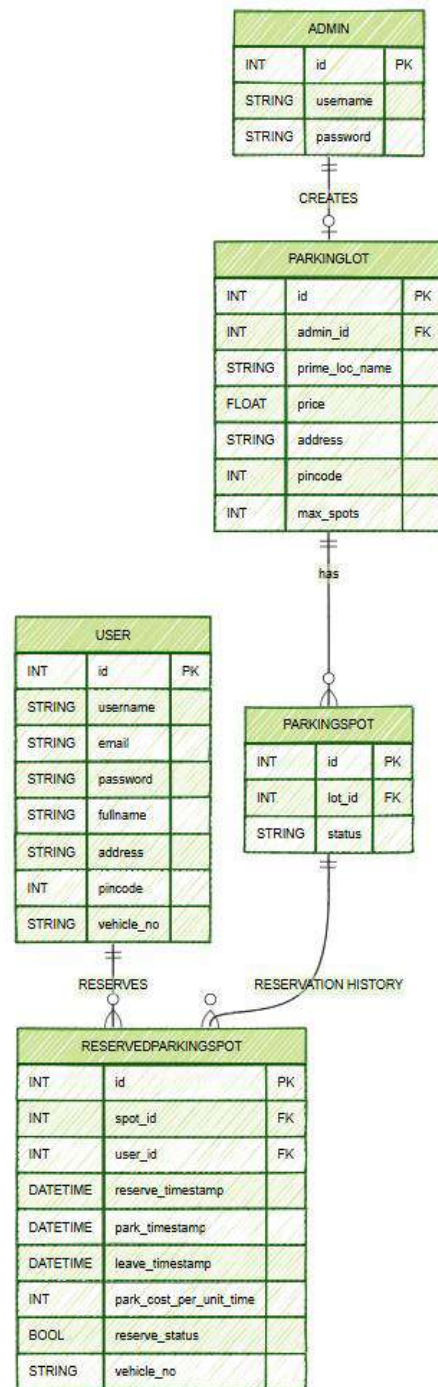
Templating

> Jinja2

Fast and secure templating engine for Python.

Purpose: Dynamically generates HTML pages by combining templates with backend data, enabling separation of business logic and presentation.

DB Schema Design



The database is structured using SQLAlchemy models, with five main tables representing the core entities of the parking management system. Below is a summary table for each table, their columns, data types, and constraints.

1. Admin Table (admins)

Column	Data Type	Constraints	Details
id	Integer	Primary Key, Autoincrement	Unique admin ID
username	String(20)	Unique, Not Null	Admin's login name
password	String(15)	Not Null	Admin password

Relations: One-to-many with parkinglots (admin can create many lots)

2. User Table (users)

Column	Data Type	Constraints	Details
id	Integer	Primary Key, Autoincrement	Unique user ID
username	String(20)	Unique, Not Null	User's username
email	String(30)	Unique, Not Null	Email
password	String(30)	Not Null	Password
fullname	String(50)	Not Null	Full name of user
address	String(200)	Not Null	User's address
pincode	Integer	Check (100000 < pincode < 1000000), Not Null	Indian pincode format
vehicle_no	String(10)	Unique	Registered vehicle number

Relations: One-to-many with reservedparkingspots (a user can create multiple reservations)

3. ParkingLot Table (parkinglots)

Column	Data Type	Constraints	Details
id	Integer	Primary Key, Autoincrement	Unique lot ID
admin_id	Integer	Foreign Key → admins.id, OnDelete CASCADE	Administrator who created the lot
prime_loc_name	String(20)	Not Null	Name/label of the lot
price	Float	Not Null	Base price for the lot
address	String(200)	Not Null	Location address
pincode	Integer	Check (100000 < pincode < 1000000), Not Null	Address pin code
max_spots	Integer	Not Null	Max capacity in terms of spots

Relations: One-to-many with parkingspots (lot contains multiple spots)

4. ParkingSpot Table (parkingspots)

Column	Data Type	Constraints	Details
id	Integer	Primary Key, Autoincrement	Unique spot ID
lot_id	Integer	Foreign Key → parkinglots.id, OnDelete CASCADE	The lot this spot belongs to
status	String(1)	Default='A', Not Null	Available/Reserved/Occupied status

Relations: One-to-many with reservedparkingspots (a spot can have reservation history)

5. ReservedParkingSpot Table (reservedparkingspots)

Column	Data Type	Constraints	Details
id	Integer	Primary Key, Autoincrement	Unique reservation entry
spot_id	Integer	Foreign Key → parkingspots.id, OnDelete CASCADE	Spot being reserved
user_id	Integer	Foreign Key → users.id, OnDelete CASCADE	User reserving the spot
reserve_timestamp	DateTime	Not Null	Reservation start time
park_timestamp	DateTime	Nullable	When the vehicle was parked
leave_timestamp	DateTime	Nullable	When the vehicle vacated
park_cost_per_unit_time	Integer	Nullable	Cost/unit time for this reservation
reserve_status	Boolean	Not Null	True=Active, False=Inactive
vehicle_no	String(10)	Not Null	Vehicle registration number

Reasons:

Majority of the database models are created according to suggested fields, with the addition of a few extra columns to implement some functionalities.

1. Foreign key linking of ParkingSpot to ParkingLot helps in creating 'n' number of 'spots' for each 'lot'. It allows to grouping and filtering spots based on location.
2. Foreign key linking of ReservedParkingSpot to ParkingSpot helps in linking reservation to the specific spot reserved. It helps prevent double-booking and checks spot availability.

3. Foreign key linking of ReservedParkingSpot to User helps in connecting each reservation to the user who made it.
4. Modularity: Different entities (users, lots, spots, reservations) were placed in separate tables to keep data organized and reduce redundancy.
5. Scalability: The design allows easy addition of new users, lots, and features (like ratings or payments) without disrupting existing structure.
6. Clarity: Separating spots and lots makes it easier to manage availability and visualize real-time status.

API Design

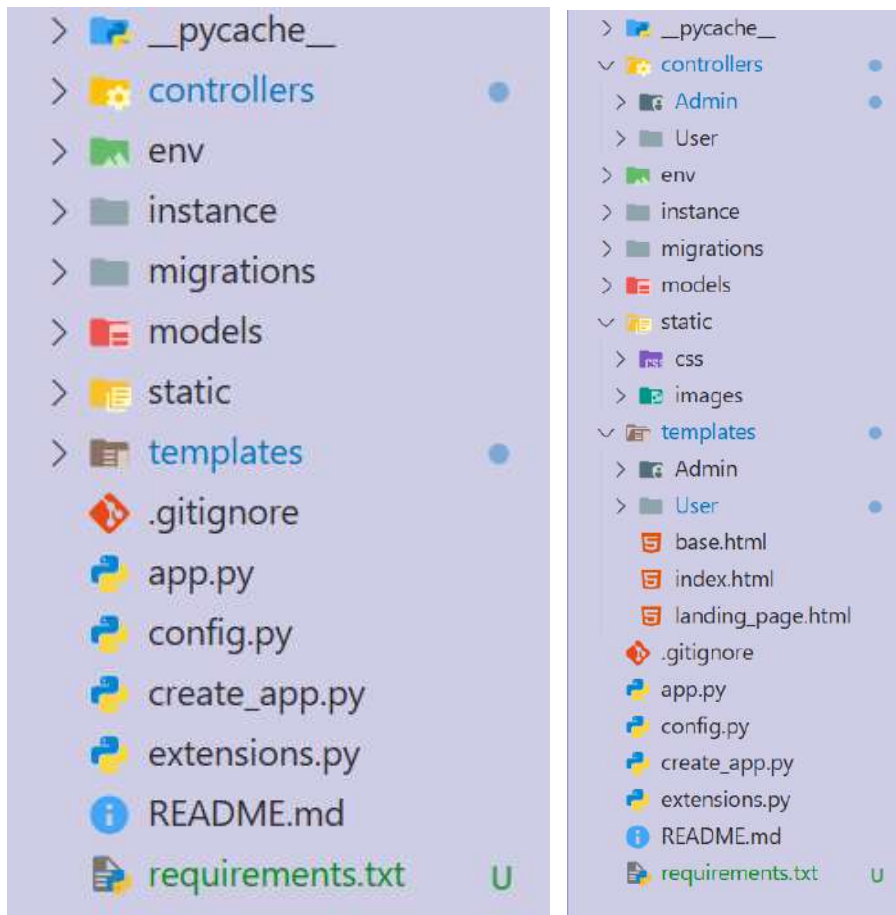
I have not used not created any API for this project

AI/LLM

1. Bootstrap/CSS - 8% (main.css file)
2. Flask – 5% (syntax)
3. SQLite/SQLAlchemy – 1% (some complex querying logic)
4. HTML+Jinja2 – 5% (loop logic and message passing)
5. Charts/Data Visualization – 2% (javascript)
6. Debugging – 10%

Architecture

According to the tutorials of the MAD-1 theory course, I have created difference folders to separate backend, frontend and database logic as well as Admin and User specific logic.



The main application is initialized in `create_app.py` and `extensions.py` handles third-party extensions like SQL Alchemy and Flask-Migrate to using Factory Pattern to prevent circular imports. Furthermore, configuration variables are stored in a `config.py` file.

Features and Implementation

1. **User authentication:** Login/logout system for users and admins using session management.
2. **Role-based dashboard:** Separate dashboards for admin and user, showing personalized data and actions.
3. **Parking spot reservation system:**
 - a. Users can search for available parking spots by location.
 - b. They can book a spot with time/date inputs.
4. **Admin control panel:**
 - a. Admins can add, update, or delete parking lots and manage spot availability.
5. **Statistics and charts:**
 - a. Admin dashboard displays visual summaries using **Chart.js** (reservation trends, status distribution, etc.).
6. **Form validation:** Implemented using Flask-WTF or HTML form checks.
7. **Flash messages** for real-time feedback (e.g., "Reservation successful").
8. **Error handling** for missing pages, database issues, or invalid inputs.
9. **Responsive UI** with Bootstrap for mobile-friendly experience.

